第 3-4 周 实验指导书

8.商品管理 API

本节实现商品的管理功能，包括图片上传、富文本编辑器的使用。

（1）上传图片

步骤 1：上传配置，在 config/default.js 文件中添加文件上传配置，端口与程序启动端口一致。

```
"upload_config":{
        "baseURL":"http://127.0.0.1:8088"
    }
```

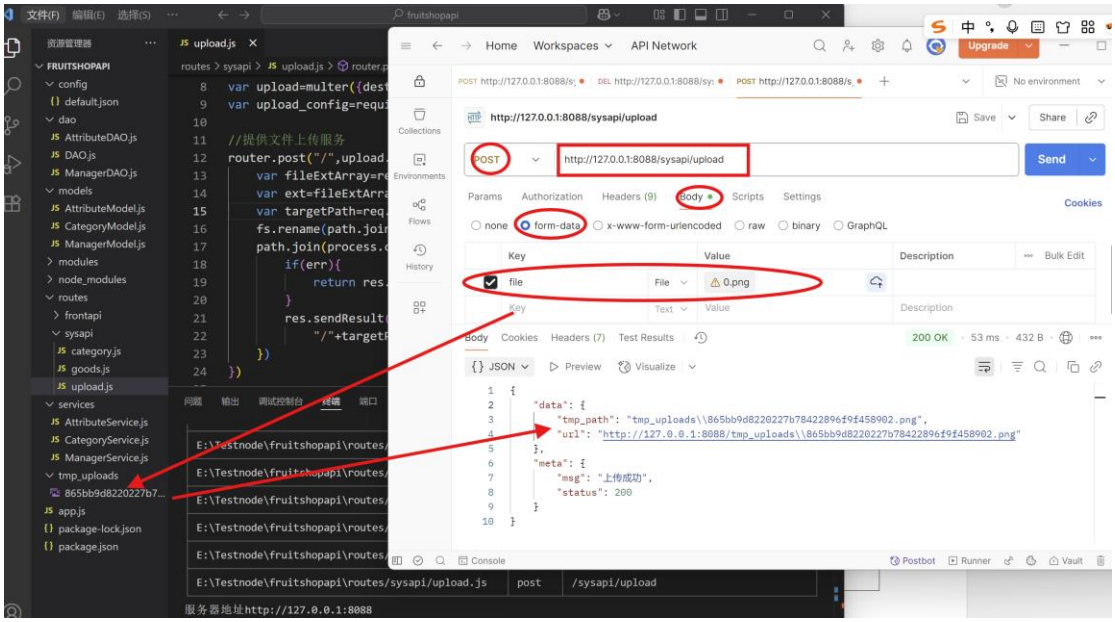
步骤 2：Multer 中间件

Multer 是一个中间件，可以处理 multipart/form-data 类型的表单数据，因此其主要用于上传文件。**multer** 中间件创建文件上传处理器的核心配置。

指令：npm i multer

步骤 3：编写上传接口：routes/sysapi/upload.js

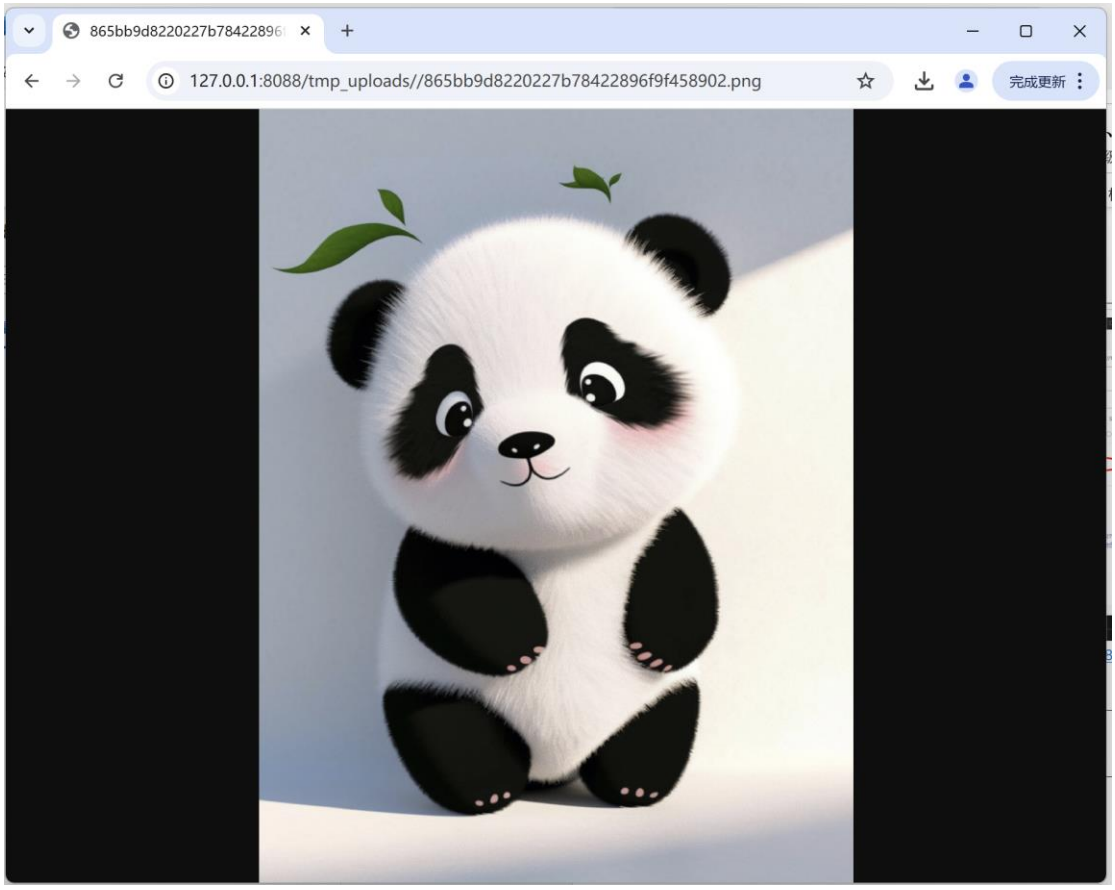
```
var express=require('express')
var router=express.Router();
var path = require("path")
var fs=require('fs')
var multer=require('multer')

//临时上传目录
var upload=multer({dest:'tmp_uploads/'}); //配置上传文件的目录
var upload_config=require('config').get("upload_config");

//提供文件上传服务
router.post("/",upload.single('file'),function(req,res,next){
    var fileExtArray=req.file.originalname.split("."); //如 1.jpg 拆分成数组
    var ext=fileExtArray[fileExtArray.length-1];      //取图片的扩展名 jpg
    var targetPath=req.file.path+"."+ext;          //目标路径:tmp_uploads\xxxxx.jpg
    fs.rename(path.join(process.cwd(),"/",req.file.path),
    path.join(process.cwd(),targetPath),function(err){
        if(err){
            return res.sendResult(null,400,"上传文件失败");
        }
        res.sendResult({"tmp_path":targetPath,"url":upload_config.get("baseURL")+
            "/"+targetPath},200,"上传成功");
    })
})
module.exports = router
```



http://127.0.0.1:8088/tmp_uploads\\865bb9d8220227b78422896f9f458902.png 开始无法访问，需要对静态资源进行挂载后方可访问。

步骤 4：挂载静态资源,app.js

```
// 挂载静态资源
app.use('/tmp_uploads',express.static('tmp_uploads'))
```



（2）添加商品

步骤 1：在 models 下创建商品表模型 GoodModel.js

```
module.exports=function(db,callback){
    //分类模型
    db.define("GoodModel",{
        goods_id:{type:'serial',key:true},
        cat_id:Number,
        goods_name:String,
        goods_price:Number,
        goods_number:Number,
        goods_weight:Number,
        goods_introduce:String,
        goods_big_logo:String,
        goods_small_logo:String,
        goods_state:Number,
        is_del:['0','1'],
        add_time:Number,
        upd_time:Number,
        delete_time:Number,
        hot_mumber:Number,
        is_promote:Boolean,
        cat_one_id:Number,
        cat_two_id:Number,
        cat_three_id:Number
    },{
        table:"goods",
        methods:{
            getGoodsCat:function(){
                return this.cat_one_id+","+this.cat_two_id+","+this.cat_three_id;
            }
        }
    });
    return callback()
}
```

步骤 2：路由层，sysapi/good.js 文件中添加商品的接口函数

```
var express=require('express')
var path=require("path");
var router=express.Router();
var GoodService=require(path.join(process.cwd(),"services/GoodService"));

//添加商品
router.post('/',function(req,res,next){
    next();
},//业务逻辑
function(req,res,next){
    var params=req.body;
    GoodService.createGood(params,function(err,newGood){
        if(err) return res.sendResult(null,400,err);
        res.sendResult(newGood,201,"创建商品成功");
    });
})
module.exports=router
```

步骤 3：创建 GoodService.js 服务层，主要功能框架如下

```
var Promise=require("bluebird");
//创建商品
module.exports.createGood=function(params,cb){
    //验证参数 & 生成数据
    generateGoodInfo(params)
    //检查商品名称
    .then(checkGoodName)
    //创建商品
    .then(createGoodInfo)
    //更新商品图片
    .then(doUpdateGoodPics)
    //获取图片
    .then(doGetAllPics)
    //创建成功
    .then(function(info){
        cb(null,info.good);
    })
    .catch(function(err){
        cb(err);
    })
}
//通过参数生成商品基本信息
function generateGoodInfo(params){
}
```

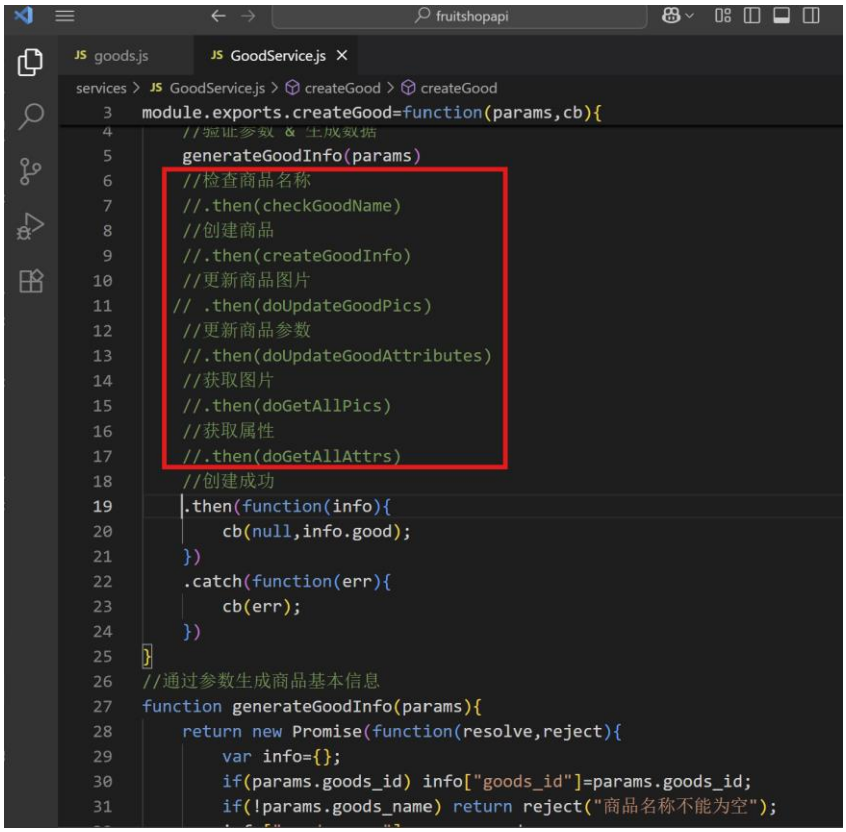
```
//检查商品名称
function checkGoodName(){

}
//创建商品
function createGoodInfo(){

}
//更新商品图片
function doUpdateGoodPics(){

}
//获取图片
function doGetAllPics(){

}
```

步骤 4：验证参数并生成数据 generateGoodInfo

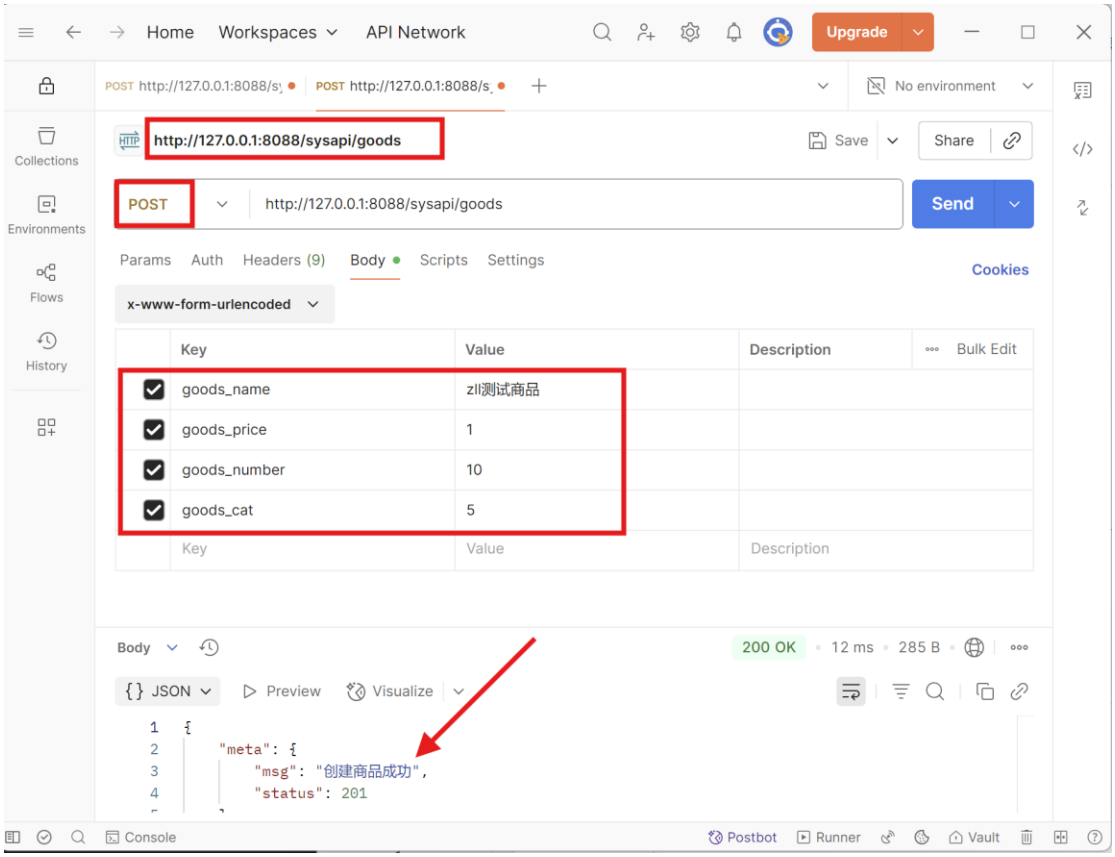
```
//通过参数生成商品基本信息
function generateGoodInfo(params){
    return new Promise(function(resolve,reject){
        var info={};
        if(params.goods_id) info["goods_id"]=params.goods_id;
        if(!params.goods_name) return reject("商品名称不能为空");
        info["goods_name"]=params.goods_name;

        if(!params.goods_price) return reject("商品价格不能为空");
        var price=parseFloat(params.goods_price);
        if(isNaN(price)||price<0) return reject("商品价格不正确");
        info["goods_price"]=price;

        if(!params.goods_number) return reject("商品数量不能为空");
        var num=parseInt(params.goods_number);
        if(isNaN(num)||num<0) return reject("商品数量不正确");
        info["goods_number"]=num;

        if(!params.goods_cat) return reject("商品没有设置所属分类");
        var cats=params.goods_cat.split(',');
        if(cats.length>0)   info["cat_one_id"]=cats[0];
        if(cats.length>1)   info["cat_two_id"]=cats[1];
        if(cats.length>2)   {
            info["cat_three_id"]=cats[2];
            info["cat_id"]=cats[2];
        }
```

```
        if(params.goods_weight) {
            weight=parseFloat(params.goods_weight);
            if(isNaN(weight)||weight<0) return reject("商品重量格式不正确");
            info["goods_weight"]=weight;
        }else{
            info["goods_weight"]=0;
        }

        if(params.goods_introduce){
            info["goods_introduce"]=params.goods_introduce;
        }

        if(params.goods_big_logo){
            info["goods_big_logo"]=params.goods_big_logo;
        }else{
            info["goods_big_logo"]="";
        }
        if(params.goods_small_logo){
            info["goods_small_logo"]=params.goods_small_logo;
        }else{
            info["goods_small_logo"]="";
        }
        if(params.goods_state){
            info["goods_state"]=params.goods_state;
        }
        //图片
        if(params.pics){
            info["pics"]=params.pics;
        }
        info["add_time"]=Date.parse(new Date())/1000;
        info["upd_time"]=Date.parse(new Date())/1000;
        info["is_del"]='0';
        if(params.hot_mumber){
            hot_num=parseInt(params.hot_mumber);
            if(isNaN(hot_num)||hot_num<0) return reject("热销品数量格式不正确");
            info["hot_mumber"]=hot_mumber;
        }else{
            info["hot_mumber"]=0;
        }
        info["is_promote"]=info["is_promote"]?info["is_promote"]:false;
        resolve(info);
    })
}
```

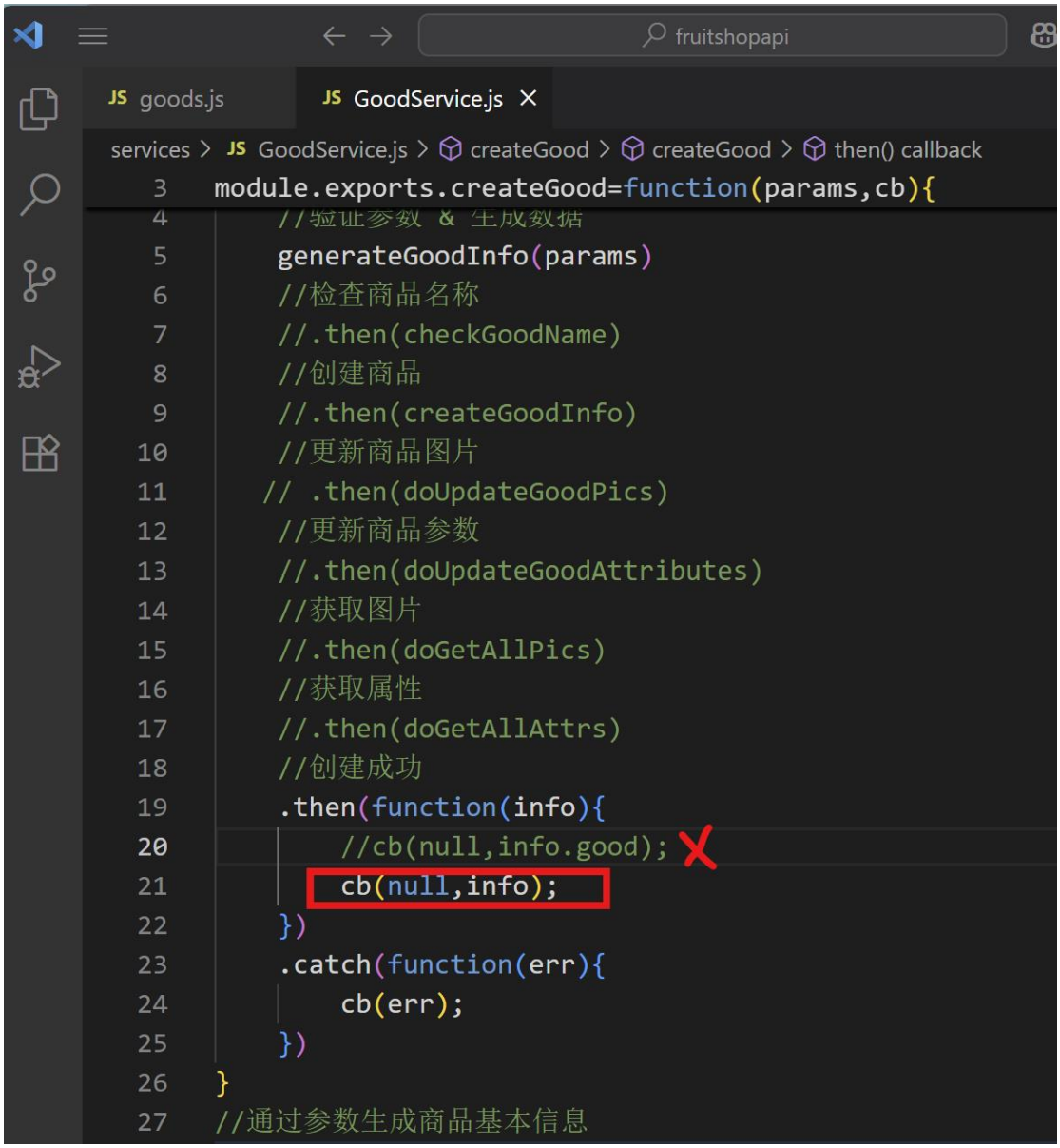
测试：先将.then.then 的信息注销一下，测试一下。

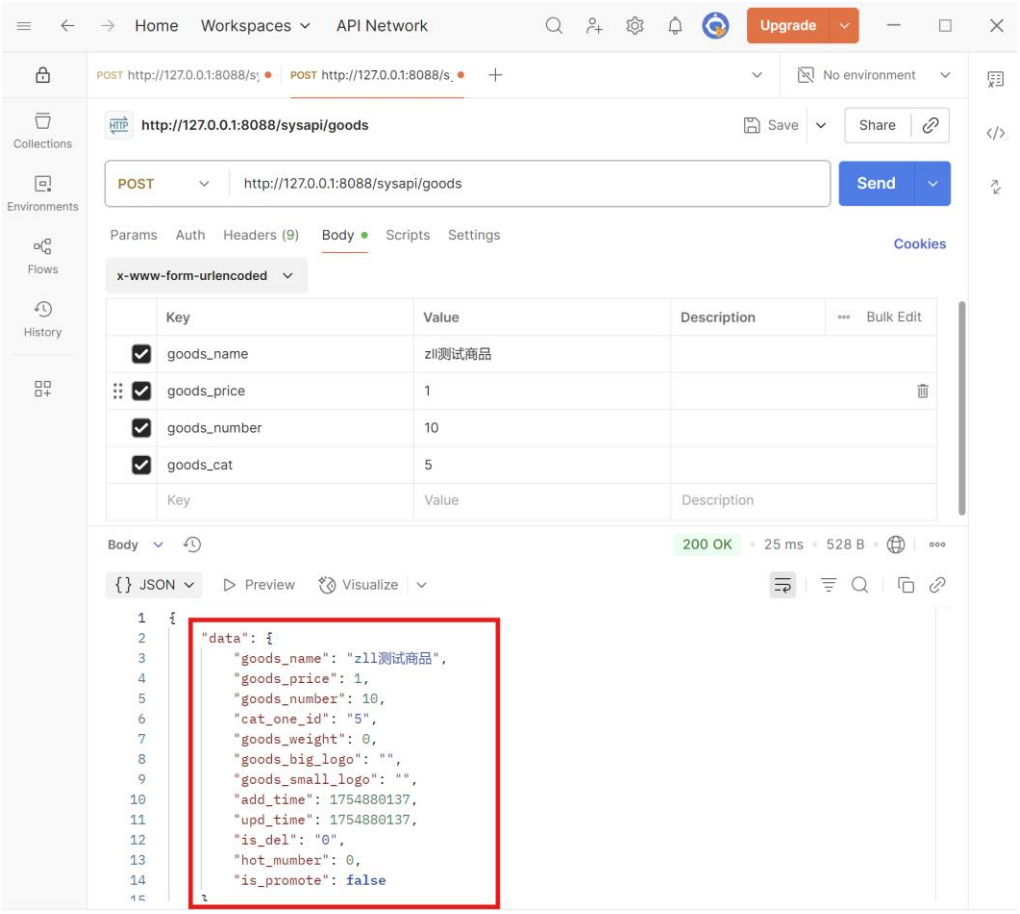


只写了 4 个必填字段



修改一下这块的值：

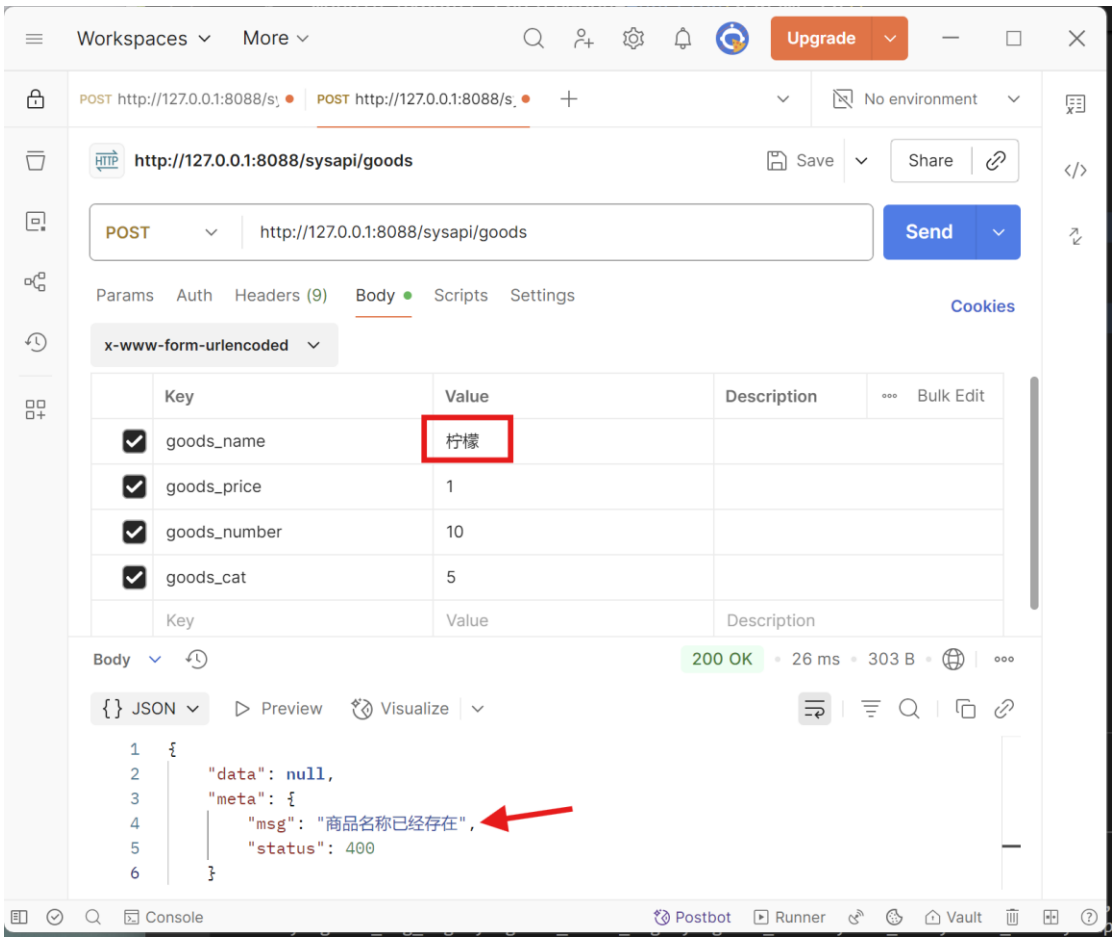

fruitshopapi
goods.js
GoodService.js
services > GoodService.js > createGood > createGood > then() callback
3 module.exports.createGood=function(params,cb){
4 //验证参数 & 生成数据
5 generateGoodInfo(params)
6 //检查商品名称
7 //.then(checkGoodName)
8 //创建商品
9 //.then(createGoodInfo)
10 //更新商品图片
11 // .then(doUpdateGoodPics)
12 //更新商品参数
13 //.then(doUpdateGoodAttributes)
14 //获取图片
15 //.then(doGetAllPics)
16 //获取属性
17 //.then(doGetAllAttrs)
18 //创建成功
19 .then(function(info){
20 //cb(null,info.good);
21 cb(null,info);
22 })
23 .catch(function(err){
24 cb(err);
25 })
26 }
27 //通过参数生成商品基本信息



步骤 5：检查商品名称是否重复

方法 checkGoodName 的代码：

```
var path=require("path")
var dao=require(path.join(process.cwd(),"dao/DAO"));
。。。。
//检查商品名称是否重复
function checkGoodName(info){
    return new Promise(function(resolve,reject){
        dao.findOne("GoodModel",{"goods_name":info.goods_name,"is_del":"0"},
            function(err,good){
                //如果有错误，直接返回错误信息
                if(err) return reject(err);
                //如果没错，但商品从数据库中未找到
                if(!good) return resolve(info);
                //如果找到，则商品属于修改
                if(good.goods_id==info.goods_id) return resolve(info);
                //除此之外说明商品已经存在，报错
                return reject("商品名称已经存在");
            }
        )
    })
}
```

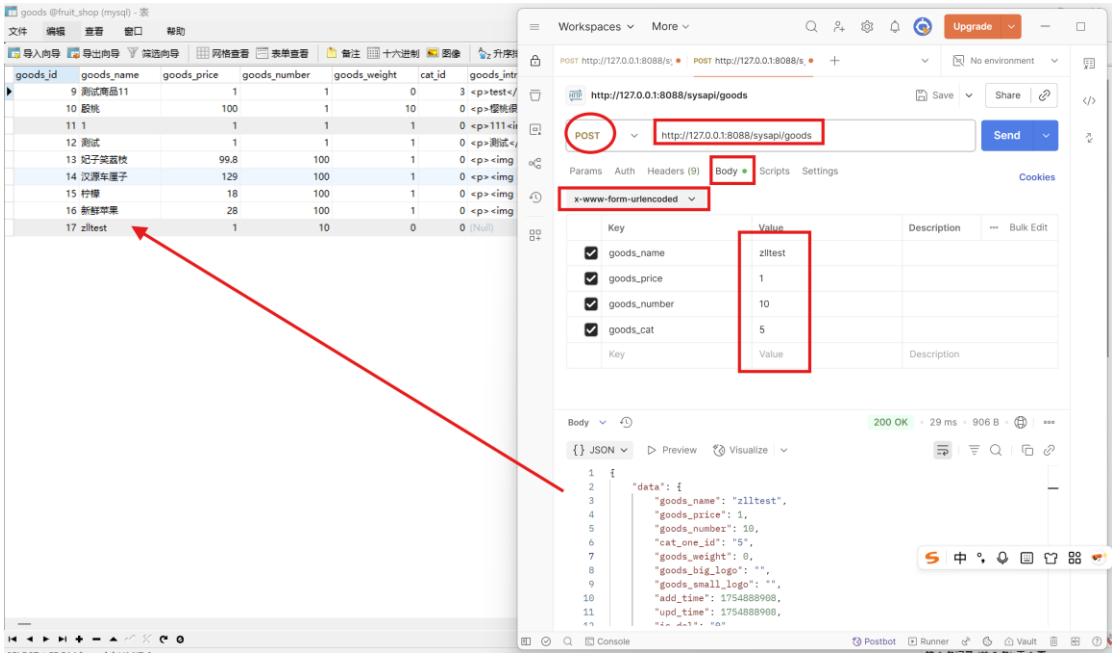


步骤 5：创建商品，要向数据库中写入商品了（注意 clone 前面的点别少了）

注意：别忘记把调用 createGoodInfo 方法前的注销符号去掉

```
var _=require('lodash')
。。。。
//创建商品，向数据库中添加商品
function createGoodInfo(info){
    return new Promise(function(resolve,reject){
        dao.create("GoodModel", _.clone(info),function(err,newGood){
            if(err) return reject("创建商品基本信息失败");
            newGood.goods_cat=newGood.getGoodsCat();
            info.good=newGood;
            return resolve(info);
        });
    });
}
```

测试：



步骤 6：更新商品图片

注意：去掉更新图片调用方法前面的注销符号

在 models 下新建 GoodPicModel.js

```
module.exports=function(db,callback){
    //分类模型
    db.define("GoodPicModel",{
        pics_id:{type:'serial',key:true},
        goods_id:Number,
        pics_big:String,
        pics_mid:String,
        pics_sma:String
    },{
        table:"goods_pics"
    });
    return callback()
}
```

在 GoodService.js 中添加 doUpdateGoodPics 方法

方法思路：从数据中取 info 的信息，如果有图片的话，更新到专门的图片数据表中。

```
var fs=require("fs")
。。。。
//更新商品图片
function doUpdateGoodPics(info){
    return new Promise(function(resolve,reject){
        var good=info.good
        if(!good.goods_id) return reject("更新商品图片失败");
        if(!info.pics) return resolve(info);
        //list 是从表中查询，如果在图片表中有 goods_id 记录就是更新
        //如果没有 goods_id 记录就新增
```

```
dao.list("GoodPicModel",
{"columns":{"goods_id":good.goods_id}},
function(err,oldpics){
    if(err) return reject("获取商品图片列表失败");
    var batchFns=[];
    var newpics=info.pics?info.pics:[];
    var newpicsKV=_.keyBy(newpics,"pics_id");
    var oldpicsKV=_.keyBy(oldpics,"pics_id");
    //保存图片集合
    //1.需要新建的图片集合
    var addNewpics=[];
    //2.需要保留的图片集合
    var reservedOldpics=[];
    //3.需要删除的图片集合
    var delOldpics=[];
    //如果提交的新数据中有老数据的 pics_id 则保留数据，否则删除数据
    _(oldpics).forEach(function (pic){
        if(newpicsKV[pic.pics_id]) reservedOldpics.push(pic);
        else delOldpics.push(pic);
    })
    //从新提交的数据中检索出需要新创建的数据
    _(newpics).forEach(function (pic){
        if(!pic.pics_id&&pic.pic)
            addNewpics.push(pic);
    })

    //开始处理商品图片数据逻辑
    //1.删除商品图片数据结合
    _(delOldpics).forEach(function(pic){
        //1.1 删除图片物理路径
    batchFns.push(removeGoodPicFile(path.join(process.cwd(),pic.pics_big)));
    batchFns.push(removeGoodPicFile(path.join(process.cwd(),pic.pics_mid)));
    batchFns.push(removeGoodPicFile(path.join(process.cwd(),pic.pics_sma)));
        //1.2 数据库删除图片数据记录
        batchFns.push(removeGoodPic(pic));
    });
    //2.处理新增图片的结合
    _(addNewpics).forEach(function (pic){
        if(!pic.pics_id&&pic.pic){
            //2.1 通过原始路片路径裁剪出需要的图片
            var src=path.join(process.cwd(),pic.pic);
            var tmp=src.split(path.sep);
            var filename=tmp[tmp.length-1];
            pic.pics_big="/uploads/goodspics/big_"+filename;
```

```
                        pic.pics_mid="/uploads/goodspics/mid_"+filename;
                        pic.pics_sma="/uploads/goodspics/sma_"+filename;

            batchFns.push(clipImage(src,path.join(process.cwd(),pic.pics_big),800,800));
            batchFns.push(clipImage(src,path.join(process.cwd(),pic.pics_mid),400,400));
            batchFns.push(clipImage(src,path.join(process.cwd(),pic.pics_sma),200,200));

                        pic.goods_id = good.goods_id;
                        //2.2 在数据库中新建数据
                        batchFns.push(createGoodPic(pic));
                    }
                });
                //如果没有任何图片操作则返回
                if(batchFns.length==0)    return resolve(info);
                //批量执行所有操作
                Promise.all(batchFns).then(function(){
                    resolve(info)
                })
                .catch(function(error){
                    if(error) return reject(error);
                });

            }
        )
        })
}
//删除图片的物理路径
function removeGoodPicFile(path){
     return new Promise(function(resolve,reject){
          fs.unlink(path,function(err,result){
               resolve();
          })
     })
}
//删除商品图片
function removeGoodPic(pic){
    return new Promise(function(resolve,reject){
          if(!pic||!pic.remove) return reject("删除商品图片记录失败");
          pic.remove(function(err){
               if(err) return reject("删除失败");
               resolve();
          })
     })
}
```

```
//剪裁商品图片
function clipImage(srcPath,savePath,newWidth,newHeight){
    return new Promise(function(resolve,reject){
        //创建读取流
        readable=fs.createReadStream(srcPath);
        //创建写入流
        writable=fs.createWriteStream(savePath);
        readable.pipe(writable);
        readable.on('end',function(){
            resolve();
        })
    })
}
// 添加商品图片到数据库
function createGoodPic(pic) {
    return new Promise(function (resolve, reject) {
        console.log("&&"+pic);
        if (!pic) return reject("图片对象不能为空");
        var GoodPicModel = dao.getModel("GoodPicModel");
        GoodPicModel.create(pic, function (err, newPic) {
            if (err) return reject("创建图片数据失败");
            resolve();
        });
    });
}
```

在 Dao.js 中增加：

```
//获取模型
module.exports.getModel = function (modelName) {
    var db = databaseModule.getDatabase();
    return db.models[modelName];
}
```

测试：图片需要上传数组对象，所以改用 JSON 格式上传，使用 body 中 raw 的 JSON 格式上传数据。一旦上传成功，在商品表中新建了一 goods_id 为 45 的记录，同时 uploads/goodspics 文件夹下会生成大中小三个图片，数据库的 goods_pics 表中也会增加 pics_id 为 45 商品的图片。注意注意：需要事先通过 http://127.0.0.1:8088/sysapi/upload 接口上传新图片到 tmp_uploads 中才可以。

第一步：清空 tmp_uploads 文件夹中的图片，调用图片上传接口，生成新的图片。

第二步：在根目录下创建 uploads/goodspics 文件夹

http://127.0.0.1:8088/sysapi/upload

Save | Share

POST http://127.0.0.1:8088/sysapi/upload **Send**

Params Auth Headers (9) Body Scripts Settings Cookies

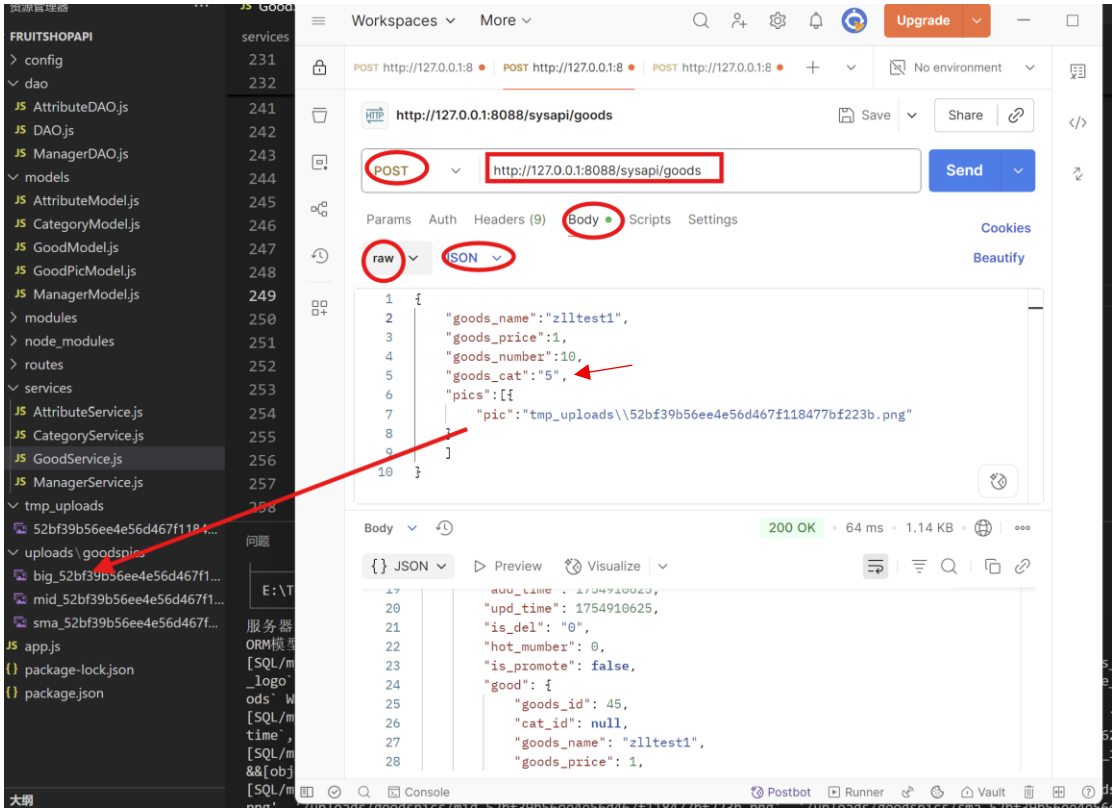
form-data

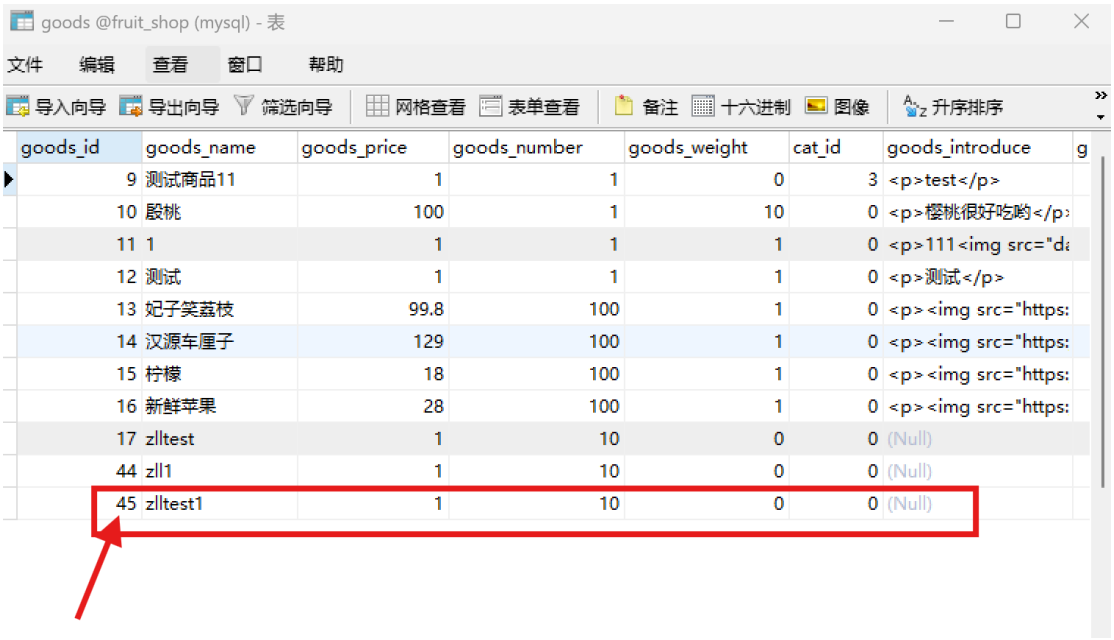
Key		Value	Description	Bulk Edit
file	File	0.png		
Key	Text	Value	Description	

Body 200 OK 92 ms 432 B

JSON Preview Visualize

```
{
    "data": {
        "tmp_path": "tmp_uploads\\52bf39b56ee4e56d467f118477bf223b.png",
        "url": "http://127.0.0.1:8088/
            tmp_uploads\\52bf39b56ee4e56d467f118477bf223b.png"
    },
    "meta": {
        "msg": "上传成功",
        "status": 200
    }
}
```



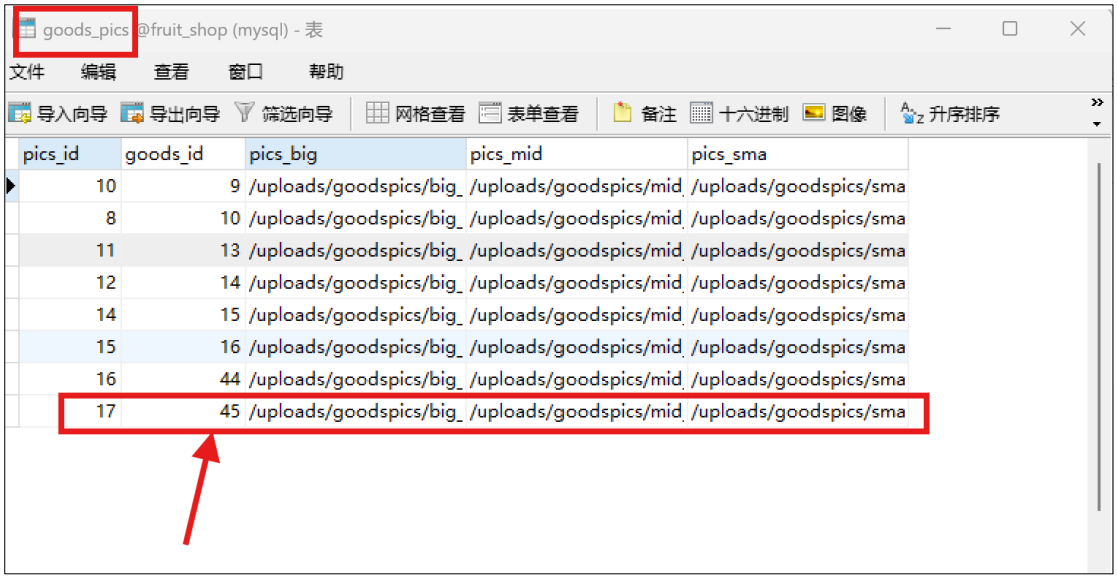


goods @fruit_shop (mysql) - 表

文件 编辑 查看 窗口 帮助

导入向导 导出向导 筛选向导 网格查看 表单查看 备注 十六进制 图像 升序排序

goods_id	goods_name	goods_price	goods_number	goods_weight	cat_id	goods_introduce	g
9	测试商品11	1	1	0	3	test	
10	殷桃	100	1	10	0	樱桃很好吃哟	
11	1	1	1	1	0	111<img src="d	
12	测试	1	1	1	0	测试	
13	妃子笑荔枝	99.8	100	1	0	<img src="https:	
14	汉源车厘子	129	100	1	0	<img src="https:	
15	柠檬	18	100	1	0	<img src="https:	
16	新鲜苹果	28	100	1	0	<img src="https:	
17	zlltest	1	10	0	0	(Null)	
44	zll1	1	10	0	0	(Null)	
45	zlltest1	1	10	0	0	(Null)	



goods_pics @fruit_shop (mysql) - 表

文件 编辑 查看 窗口 帮助

导入向导 导出向导 筛选向导 网格查看 表单查看 备注 十六进制 图像 升序排序

pics_id	goods_id	pics_big	pics_mid	pics_sma
10	9	/uploads/goodspics/big_	/uploads/goodspics/mid	/uploads/goodspics/sma
8	10	/uploads/goodspics/big_	/uploads/goodspics/mid	/uploads/goodspics/sma
11	13	/uploads/goodspics/big_	/uploads/goodspics/mid	/uploads/goodspics/sma
12	14	/uploads/goodspics/big_	/uploads/goodspics/mid	/uploads/goodspics/sma
14	15	/uploads/goodspics/big_	/uploads/goodspics/mid	/uploads/goodspics/sma
15	16	/uploads/goodspics/big_	/uploads/goodspics/mid	/uploads/goodspics/sma
16	44	/uploads/goodspics/big_	/uploads/goodspics/mid	/uploads/goodspics/sma
17	45	/uploads/goodspics/big_	/uploads/goodspics/mid	/uploads/goodspics/sma

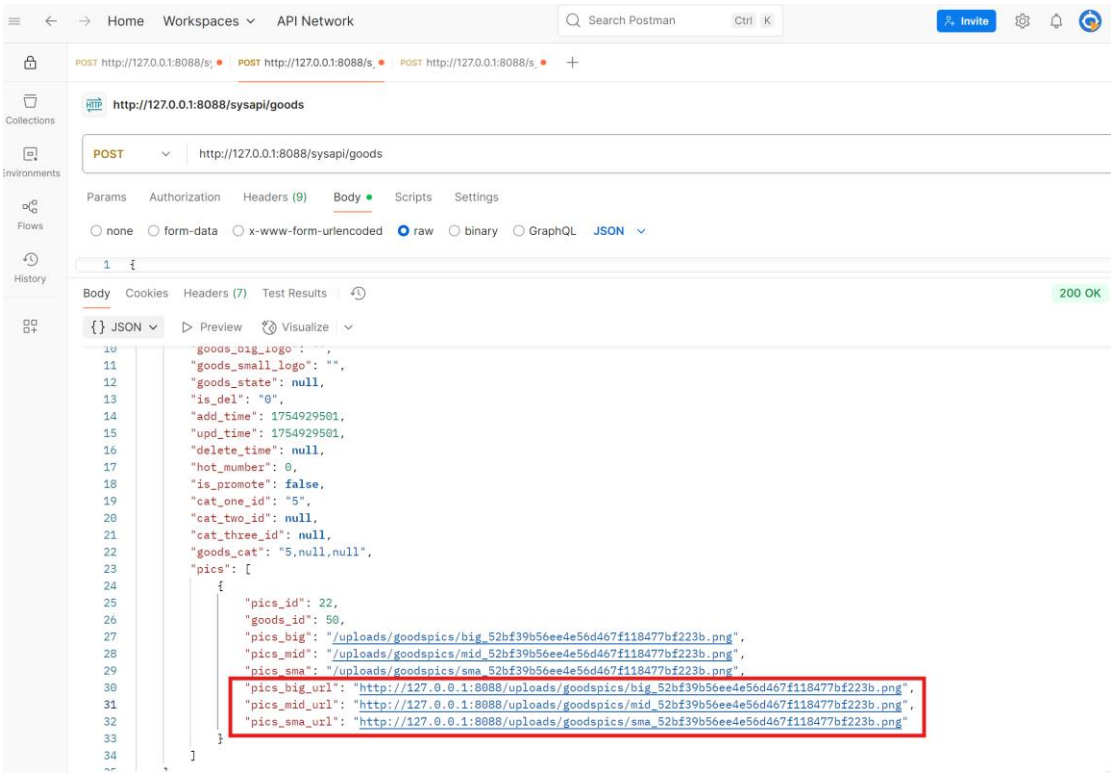
步骤 7：挂载图片

在前面执行的所有代码，图片还没有加载出来。这个步骤是把图片从数据库数据表 goods_pics 中取出来，拼接上 http 等真实地址，存储到商品表的记录的 pics 属性上。

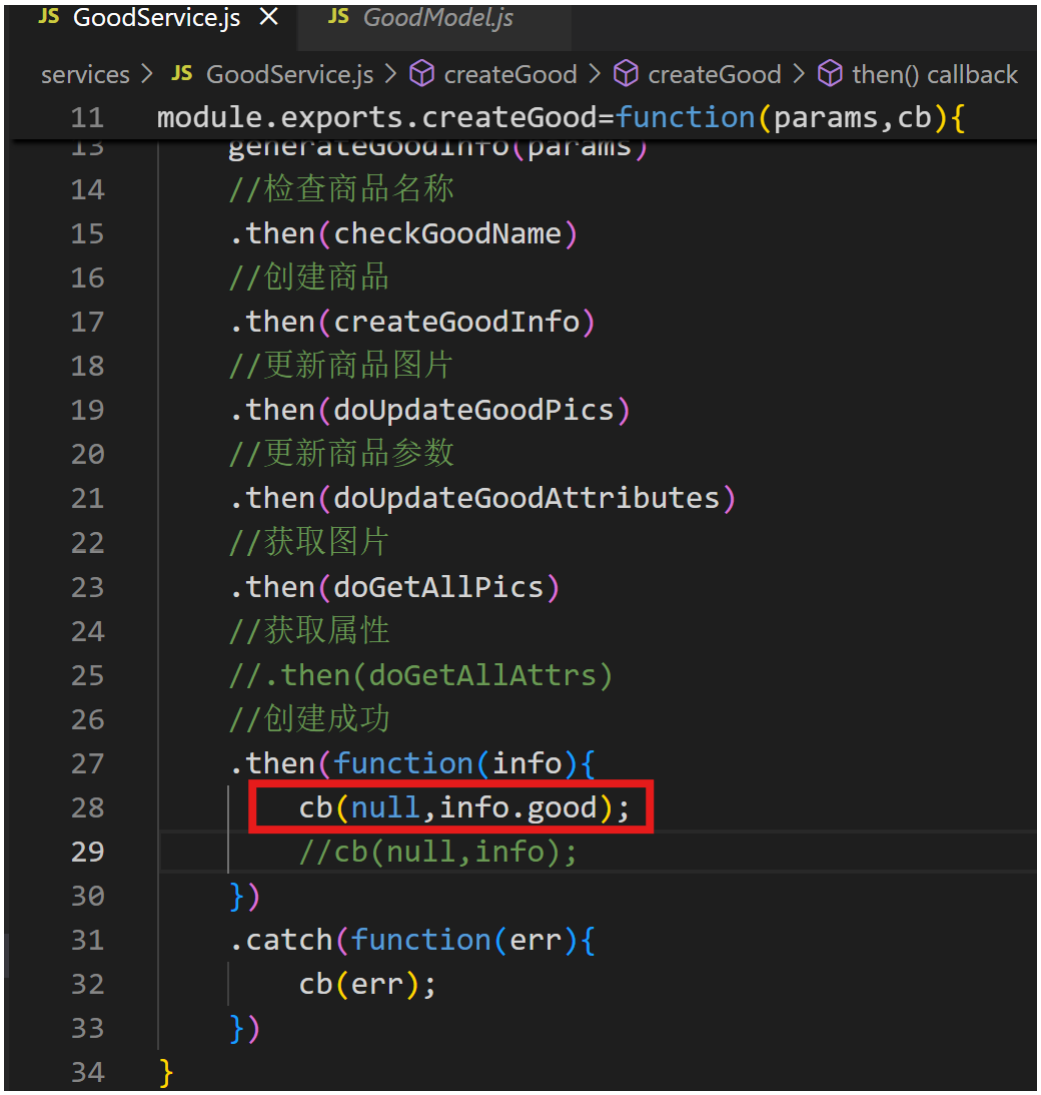
```
var upload_config = require('config').get("upload_config");
。。。。
//挂载图片
function doGetAllPics(info) {
    return new Promise(function (resolve, reject) {
        var good = info.good;
        if (!good.goods_id) return reject("获取商品图片必须先获取商品信息");
        // 3. 组装最新的数据挂载在“info”中“good”对象下
        dao.list("GoodPicModel", { "columns": { "goods_id": good.goods_id } },
            function (err, goodPics) {
                if (err) return reject("获取所有商品图片列表失败");
```

```
                    _(goodPics).forEach(function (pic) {
                        if (pic.pics_big.indexOf("http") == 0) {
                            pic.pics_big_url = pic.pics_big;
                        } else {
                            pic.pics_big_url = upload_config.get("baseURL") +
pic.pics_big;
                        }

                        if (pic.pics_mid.indexOf("http") == 0) {
                            pic.pics_mid_url = pic.pics_mid;
                        } else {
                            pic.pics_mid_url = upload_config.get("baseURL") +
pic.pics_mid;
                        }
                        if (pic.pics_sma.indexOf("http") == 0) {
                            pic.pics_sma_url = pic.pics_sma;
                        } else {
                            pic.pics_sma_url = upload_config.get("baseURL") +
pic.pics_sma;
                        }
                    });
                    info.good.pics = goodPics;
                    resolve(info);
                });
    });
}
```

测试：删掉商品表 goods、图片表 goods_pics 中 goods_id 为 47 的记录，重新测试，可以看到 pics。



注意：下图未知的 cb，这样写 postman 中的数据更简洁。



（3）获取商品列表

步骤 1：数据访问层

在 DAO.js 中添加 countByConditions 方法

```
//根据条件查询
module.exports.countByConditions = function (modelName,conditions,cb) {
    var db = databaseModule.getDatabase();
    var model=db.models[modelName];
    if(!model) return cb("模型不存在",null);
    var resultCB=function(err,count){
        if(err) return cb("查询失败",null);
        cb(null,count);
    }
    if(conditions){
        if(conditions["columns"]){
            model=model.count(conditions["columns"],resultCB);
        }else{
            model=model.count(resultCB);
```

```
            }
        }else{
            model=model.count(resultCB);
        }
}
```

步骤 2：业务层

在 GoodService.js 文件中添加 getAllGoods 方法

```
var orm=require("orm")
。。。。
//获取商品列表
module.exports.getAllGoods=function(params,cb){
    var conditions={};
    if(!params.pagenum||params.pagenum<=0) return cb("pagenum 参数错误")
    if(!params.pagesize||params.pagesize<=0) return cb("pagesize 参数错误")
    conditions["columns"]={};
    if(params.query){
        conditions["columns"]["goods_name"]=orm.like("%"+params.query+"%");
    }
    conditions["columns"]["is_del"]='0';
    dao.countByConditions("GoodModel",conditions,function(err,count){
        if(err) return cb(err);
        pagesize=params.pagesize;
        pagenum=params.pagenum;
        pageCount=Math.ceil(count/pagesize);
        offset=(pagenum-1)*pagesize;
        if(offset>=count) offset=count;
        limit=pagesize;
        //构建条件
        conditions["offset"]=offset; //跳过的记录数量
        conditions["limit"]=limit;    //返回的最大记录数量
       //设置查询时只能返回特定的字段
        conditions["only"]=["goods_id","goods_name","goods_price","goods_weight"
,"goods_state",
        "add_time","goods_number","upd_time","hot_number","is_promote"];
        conditions["order"]="-add_time";

        dao.list("GoodModel",conditions,function(err,goods){
            if(err) return cb(err);
            var resultDta={};
            resultDta["total"]=count;
            resultDta["pagenum"]=pagenum;
             //_.map 遍历 goods 数组，_.omit 过滤掉（删除掉）一些字段
            resultDta["goods"]=_.map(goods,function(good){
                return
```

```
_.omit(good,"goods_introduce","is_del","goods_big_logo","goods_small_logo","delete_
time");
                    });
                    cb(null,resultDta);
            });
    });
}
```

步骤 3：路由层

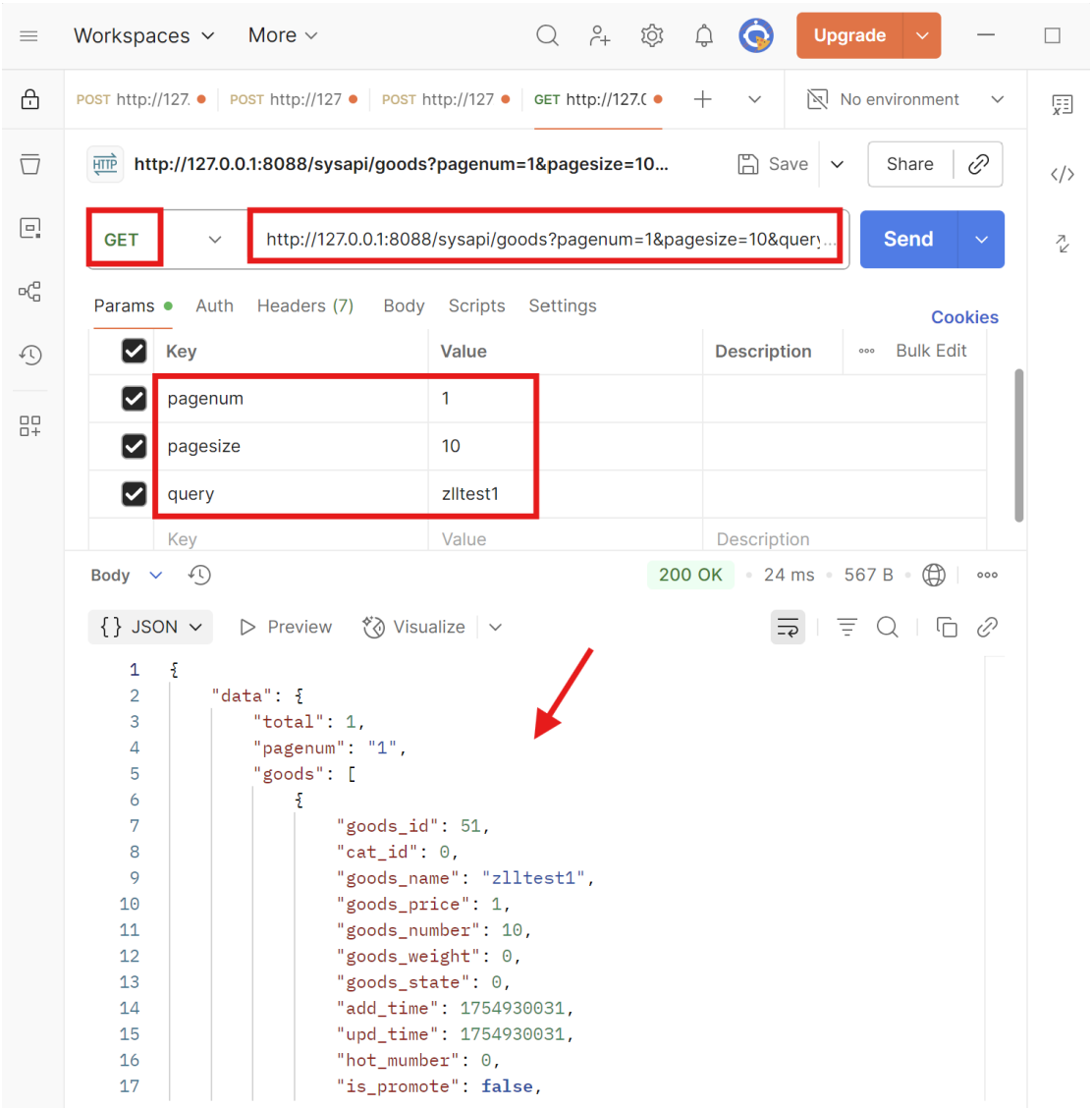
在 goods 文件中添加获取商品列表的接口方法

```
var express=require('express')
var path=require("path");
var router=express.Router();
var GoodService=require(path.join(process.cwd(),"services/GoodService"));

//添加商品
router.post('/',function(req,res,next){
    next();
},//业务逻辑
function(req,res,next){
    var params=req.body;
    GoodService.createGood(params,function(err,newGood){
        if(err) return res.sendResult(null,400,err);
        res.sendResult(newGood,201,"创建商品成功");
    });
})

//返回商品列表
router.get('/',function(req,res,next){
    if(!req.query.pagenum||req.query.pagenum<=0)
        return res.sendResult(null,400,"pagenum 参数错误");
    if(!req.query.pagesize||req.query.pagesize<=0)
        return res.sendResult(null,400,"pagesize 参数错误");
    next();
},//业务逻辑
function(req,res,next){
    var conditions={
        "pagenum":req.query.pagenum,
        "pagesize":req.query.pagesize
    };
    if(req.query.query)    conditions["query"]=req.query.query;

    GoodService.getAllGoods(conditions,function(err,result){
        if(err) return res.sendResult(null,400,err);
```

```
            res.sendResult(result,201,"获取成功");
        });
})
module.exports=router
```

步骤 4：测试

在 params 中传参数。



（4）删除商品

步骤 1：服务层

在 GoodService.js 文件中添加删除商品的 deleteGood 方法

```
//删除商品
module.exports.deleteGood=function(id,cb){
    if(!id) return cb("商品 ID 不能为空");
    if(isNaN(id)) return cb("商品 ID 必须为数字");
    dao.update("GoodModel",id,{
        "is_del":"1",
```

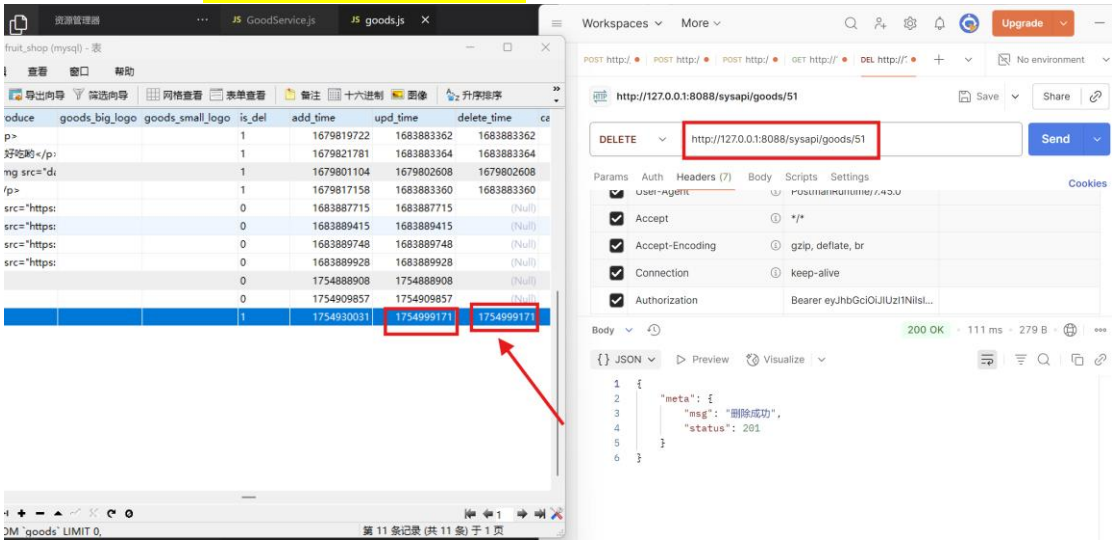
```
            "delete_time":Date.parse(new Date())/1000,
            "upd_time":Date.parse(new Date())/1000
        },
        function(err){
            if(err) return cb(err)
                cb(null);
        }
        );
}
```

步骤 2：路由层

在 good.js 文件中添加删除商品的方法 deleteGood 方法

```
//删除商品
router.delete("/:id",
    //参数验证
    function(req,res,next){
        if(!req.params.id) return res.sendResult(null,400,"商品 ID 不能为空");
        if(isNaN(req.params.id)) return res.sendResult(null,400,"商品 ID 必须为数字");
        next();
    },//业务逻辑
    function(req,res,next){
        GoodService.deleteGood(req.params.id,function(err,result){
            if(err) return res.sendResult(null,400,"删除失败");
            res.sendResult(result,201,"删除成功");
        });
})
```

步骤 3：测试，记录的删除时间添加上了。



（5）修改商品

步骤 1：服务层

在 GoodService.js 文件中添加修改商品的 updateGood 方法

```
//更新商品
module.exports.updateGood=function(id,params,cb){
```

```
    params.goods_id=id;
    generateGoodInfo(params)
    //检查商品名称
    .then(checkGoodName)
    //修改商品
    .then(updateGoodInfo)
    //更新商品图片
    .then(doUpdateGoodPics)
    //获取图片
    .then(doGetAllPics)
    //创建成功
    .then(function(info){
        cb(null,info.good);
    })
    .catch(function(err){
        cb(err);
    });
}
//修改商品信息
function updateGoodInfo(info){
    return new Promise(function (resolve, reject){
        if(!info.goods_id) return reject("商品 ID 不存在");
        dao.update("GoodModel",info.goods_id,_.clone(info),function(err,newGood){
            if(err) return reject("更新商品基本信息失败");
            info.good=newGood;
            return resolve(info);
        })
    })
}
```

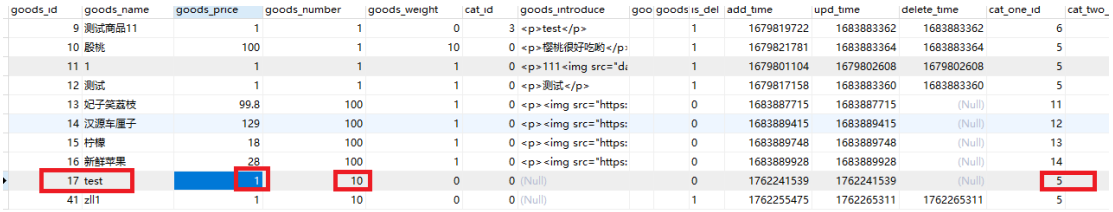
步骤 2：路由层

在 goods.js 文件中添加更新商品接口

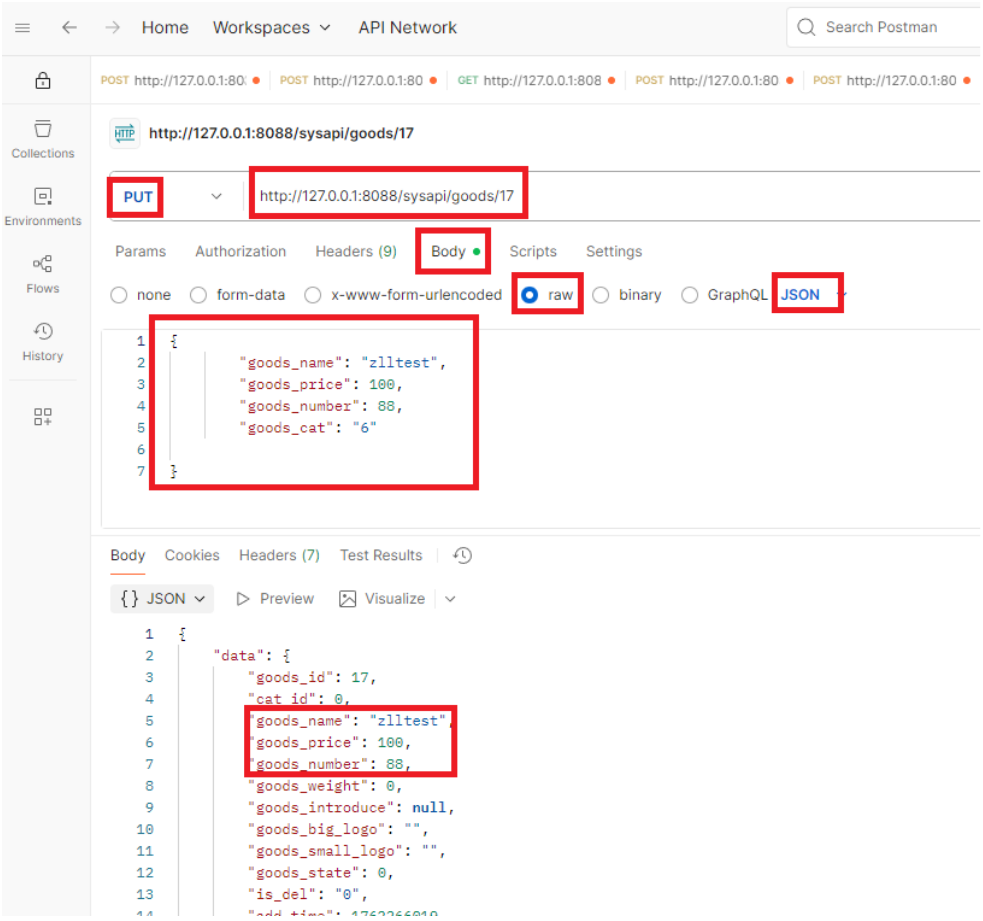
```
//更新商品
router.put("/:id",
    //参数验证
    function(req,res,next){
        if(!req.params.id) return res.sendResult(null,400,"商品 ID 不能为空");
        if(isNaN(req.params.id)) return res.sendResult(null,400,"商品 ID 必须为数字");
        next();
    },//业务逻辑
    function(req,res,next){
        var params=req.body;
        GoodService.updateGood(req.params.id,params,function(err,newGood){
            if(err) return res.sendResult(null,400,err);
            res.sendResult(newGood,200,"更新商品成功");
```

```
        });
})
```

步骤 3：测试



goods_id	goods_name	goods_price	goods_number	goods_weight	cat_id	goods_introduce	goo	goods is_del	add_time	upd_time	delete_time	cat_one_id	cat_two_
9	测试商品11	1	1	0	3	test		1	1679819722	1683883362	1683883362	6	
10	殷桃	100	1	10	0	樱桃很好吃哟		1	1679821781	1683883364	1683883364	5	
11	1	1	1	1	0	111<img src="da		1	1679801104	1679802608	1679802608	5	
12	测试	1	1	1	0	测试		1	1679817158	1683883360	1683883360	5	
13	妃子笑荔枝	99.8	100	1	0	<img src="https:		0	1683887715	1683887715	(Null)	11	
14	汉源车厘子	129	100	1	0	<img src="https:		0	1683889415	1683889415	(Null)	12	
15	柠檬	18	100	1	0	<img src="https:		0	1683889748	1683889748	(Null)	13	
16	新鲜苹果	28	100	1	0	<img src="https:		0	1683889928	1683889928	(Null)	14	
17	test	1	10	0	0	(Null)		0	1762241539	1762241539	(Null)	5	
41	zll1	1	10	0	0	(Null)		1	1762255475	1762265311	1762265311	5	



goods_id	goods_name	goods_price	goods_number	goods_weight	cat_id	goods_introduce	goo	goods is_del	add_time	upd_time	delete_time	cat_one_id	cat_two
9	测试商品11	1	1	0	3	test		1	1679819722	1683883362	1683883362	6	
10	殷桃	100	1	10	0	樱桃很好吃哟		1	1679821781	1683883364	1683883364	5	
11	1	1	1	1	0	111<img src="da		1	1679801104	1679802608	1679802608	5	
12	测试	1	1	1	0	测试		1	1679817158	1683883360	1683883360	5	
13	妃子笑荔枝	99.8	100	1	0	<img src="https:		0	1683887715	1683887715	(Null)	11	
14	汉源车厘子	129	100	1	0	<img src="https:		0	1683889415	1683889415	(Null)	12	
15	柠檬	18	100	1	0	<img src="https:		0	1683889748	1683889748	(Null)	13	
16	新鲜苹果	28	100	1	0	<img src="https:		0	1683889928	1683889928	(Null)	14	
17	zlltest	100	88	0	0	(Null)		0	1762266019	1762266019	(Null)	6	
41	zll1	1	10	0	0	(Null)		1	1762255475	1762265311	1762265311	5	

（6）获取商品详情

步骤 1：服务层

在 GoodService.js 文件中添加获取商品信息的 getGoodById 方法

```
//通过商品 ID 获取商品详情
module.exports.getGoodById=function(id,cb){
    getGoodInfo({"goods_id":id})
    .then(doGetAllPics)
```

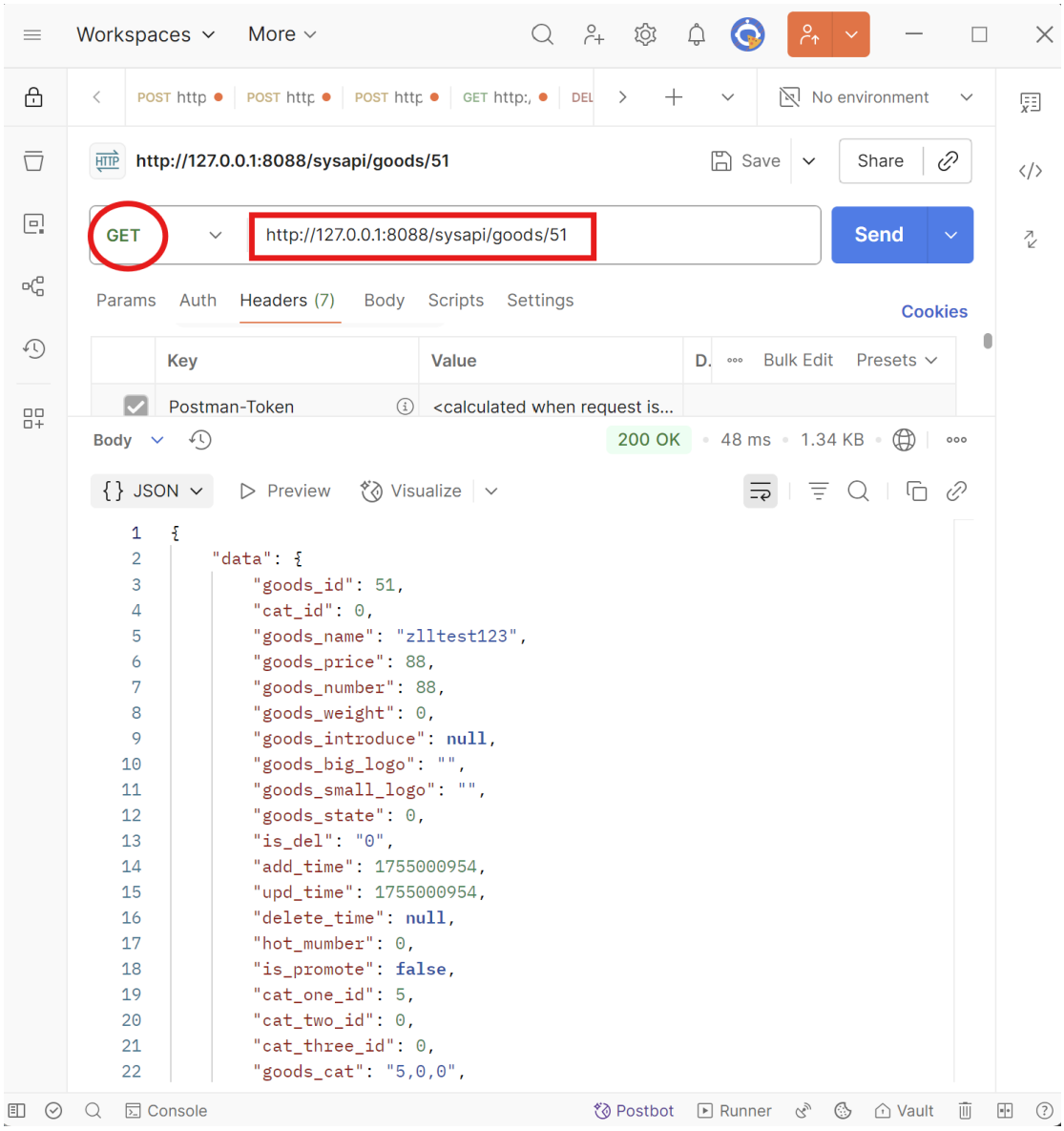
```
    .then(function(info){
        cb(null,info.good)
    })
    .catch(function(err){
        cb(err)
    })
}
//获取商品对象
function getGoodInfo(info){
    return new Promise(function(resolve,reject){
        if(!info||!info.goods_id||isNaN(info.goods_id)) return reject("商品 ID 格式不正
确");
        dao.show("GoodModel",info.goods_id,function(err,good){
            if(err) return reject("获取商品基本信息失败");
            good.goods_cat=good.getGoodsCat();
            info["good"]=good;
            return resolve(info);
        })
    })
}
```

步骤 2：路由层

在 good.js 文件中添加根据商品 ID 获取商品详情的接口

```
//通过商品 ID 获取商品详情
router.get("/:id",
    //参数验证
    function(req,res,next){
        if(!req.params.id) return res.sendResult(null,400,"商品 ID 不能为空");
        if(isNaN(req.params.id)) return res.sendResult(null,400,"商品 ID 必须为数字");
        next();
    },//业务逻辑
    function(req,res,next){
        GoodService.getGoodById(req.params.id,function(err,good){
            if(err) return res.sendResult(null,400,err);
            res.sendResult(good,200,"获取成功");
        });
})
```

步骤 3：测试


Workspaces
More
No environment
http://127.0.0.1:8088/sysapi/goods/51
Save
Share
GET
http://127.0.0.1:8088/sysapi/goods/51
Send
Params
Auth
Headers (7)
Body
Scripts
Settings
Cookies
Key
Value
Bulk Edit
Presets
Postman-Token
<calculated when request is...
Body
200 OK
48 ms
1.34 KB
JSON
Preview
Visualize
{
"data": {
"goods_id": 51,
"cat_id": 0,
"goods_name": "zlltest123",
"goods_price": 88,
"goods_number": 88,
"goods_weight": 0,
"goods_introduce": null,
"goods_big_logo": "",
"goods_small_logo": "",
"goods_state": 0,
"is_del": "0",
"add_time": 1755000954,
"upd_time": 1755000954,
"delete_time": null,
"hot_mumber": 0,
"is_promote": false,
"cat_one_id": 5,
"cat_two_id": 0,
"cat_three_id": 0,
"goods_cat": "5,0,0",
Console
Postbot
Runner
Vault